Guía Práctica de Laboratorio: Implementación de Control de Acceso y Protección de Rutas con Ionic, Angular Standalone y Functional Guards

- **Módulo Profesional:** Desarrollo de Interfaces (Código: 0488) — 2º Curso DAM.
- **Alineación Curricular (Comunidad de Madrid):**
  - **RA 1:** Confecciona interfaces de usuario identificando y aplicando patrones arquitectónicos y optimizando la experiencia de usuario.
    - **CE 1.b:** Se han identificado y aplicado patrones de diseño arquitectónicos adecuados (separación de responsabilidades, MVVM).
    - **CE 1.c:** Se ha implementado un sistema de enrutamiento modular y navegación estructurada.
    - **CE 1.d:** Se han establecido mecanismos de control de acceso y protección de rutas mediante *guards*.
    - **CE 1.e:** Se ha gestionado y asegurado la persistencia del estado de autenticación en el cliente.

1. Base Teórica Técnica

1.1. Arquitectura de Navegación y Enfoque Funcional

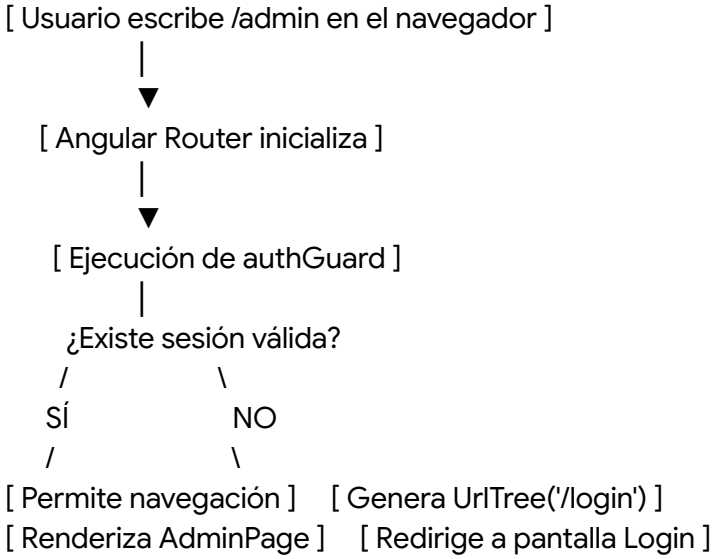
En las arquitecturas modernas de Angular (v16+), el sistema de enrutamiento abandonó las interfaces de clases heredadas (CanActivate) para dar paso a los **Functional Guards** (CanActivateFn).

- Una guarda funcional es una función pura que se ejecuta en el pipeline del enrutador antes de resolver la carga de un componente.
- Utiliza la función inject() dentro del contexto de inyección para obtener dependencias (AuthService, Router) sin necesidad de constructores.
- Devuelve un valor booleano (boolean), una promesa, un observable o, como estándar de calidad UX, un objeto **UrlTree** para provocar una redirección atómica e instantánea.

1.2. Ciclo de Vida de la Navegación y Persistencia del Estado

Cuando un usuario introduce directamente una URL en la barra de direcciones del navegador (por ejemplo, http://localhost:8100/admin), se reinicia la aplicación en memoria (recarga total del contexto de JavaScript). Para evitar que un usuario autenticado pierda su sesión o que un usuario anónimo engañe al estado en memoria:

1. El estado de la sesión debe sincronizarse con un mecanismo de persistencia local en el navegador del cliente (Storage API).
2. El enrutador invoca a la guarda asignada a la ruta solicitada antes de montar la vista.
3. La guarda verifica el estado en el servicio (que a su vez consulta la persistencia síncrona). Si la validación falla, cancela la navegación hacia la ruta protegida y devuelve un UrlTree hacia /login.



2. Implementación Práctica Paso a Paso

Paso 1: Creación del Proyecto Ionic con Standalone Components

Ejecuta en tu terminal el asistente de Ionic CLI asegurando una plantilla limpia (blank) basada en componentes independientes de Angular:

```
Bash
ionic start ionic-auth-guard blank --type=angular-standalone --no-interactive
cd ionic-auth-guard
```

Genera los elementos necesarios mediante el CLI:

```
Bash
# Servicio de autenticación
ionic g service core/services/auth

# Guarda funcional
ionic g guard core/guards/auth --type=canActivate

# Vistas
ionic g page pages/login
ionic g page pages/admin
```

Paso 2: Servicio de Autenticación y Gestión de Persistencia (src/app/core/services/auth.service.ts)

La sesión se almacena en el **sessionStorage** o **localStorage** del navegador.

- Usaremos sessionStorage para este laboratorio didáctico: los datos persisten durante la sesión del navegador (pestaña activa) y se destruyen al cerrar la pestaña, facilitando las pruebas de ciclo de vida.
- El estado interno se gestiona reactivamente mediante **Angular Signals**.

TypeScript

```
import { Injectable, signal, computed } from '@angular/core';

export interface UserSession {
  username: string;
  role: 'admin' | 'user';
  isAuthenticated: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  // Clave empleada en la Web Storage API
  private readonly STORAGE_KEY = 'DAM2_AUTH_SESSION';

  // Signal reactivo para el estado en memoria
  private _currentUser = signal<UserSession null |>(this.readSessionFromStorage());

  // Selectores computados de solo lectura expuestos a la aplicación
  public currentUser = this._currentUser.asReadonly();
  public isAuthenticated = computed(() => this._currentUser()?.isAuthenticated ?? false);

  /**
   * Valida credenciales requeridas: admin / 123
   */
  public login(user: string, pass: string): boolean {
    if (user === 'admin' && pass === '123') {
      const sessionData: UserSession = {
        username: user,
        role: 'admin',
        isAuthenticated: true
      };
```

```
      // 1. Guardar en memoria reactiva
      this._currentUser.set(sessionData);

      // 2. Persistir en el navegador (Web Storage API)
      sessionStorage.setItem(this.STORAGE_KEY, JSON.stringify(sessionData));

      return true;
    }
    return false;
  }

  /**
   * Destruye la sesión activa en memoria y en almacenamiento local
   */
  public logout(): void {
    sessionStorage.removeItem(this.STORAGE_KEY);
    this._currentUser.set(null);
  }

  /**
   * Recupera la sesión al refrescar la página
   */
  private readSessionFromStorage(): UserSession | null {
    const rawData = sessionStorage.getItem(this.STORAGE_KEY);
    if (!rawData) {
      return null;
    }
    try {
      return JSON.parse(rawData) as UserSession;
    } catch {
      sessionStorage.removeItem(this.STORAGE_KEY);
      return null;
    }
  }
}
```

Paso 3: Definición del Functional Guard (src/app/core/guards/auth.guard.ts)

La guarda evalúa si la sesión es válida. En lugar de retornar un booleano plano, redirige de forma segura devolviendo un UrlTree.

```
TypeScript
import { inject } from '@angular/core';
import { CanActivateFn, Router, UrlTree } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state): boolean | UrlTree => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Verificación de estado reactivo (nutrido desde memoria o Web Storage)
  if (authService.isAuthenticated()) {
    return true;
  }

  // Si no está autenticado, redirigir atómicamente a la página de login
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

Paso 4: Configuración de Rutas de la Aplicación (src/app/app.routes.ts)

Configura la carga perezosa (*lazy loading*) vinculando la guarda de seguridad en la ruta administrativa:

```
TypeScript
import { Routes } from '@angular/router';
import { authGuard } from './core/guards/auth.guard';

export const routes: Routes = [
 {
  path: '',
  redirectTo: 'login',
  pathMatch: 'full',
 },
 {
  path: 'login',
  loadComponent: () => import('./pages/login/login.page').then(m => m.LoginPage)
 },
 {
  path: 'admin',
  canActivate: [authGuard], // Protección mediante Functional Guard
  loadComponent: () => import('./pages/admin/admin.page').then(m => m.AdminPage)
 },
 {
  path: '**',
  redirectTo: 'login'
 }
];
```

Paso 5: Vista y Lógica de Inicio de Sesión

src/app/pages/login/login.page.html

HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Control de Acceso</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-card>
    <ion-card-header>
      <ion-card-title>Iniciar Sesión</ion-card-title>
      <ion-card-subtitle>Introduce tus credenciales del sistema</ion-card-subtitle>
    </ion-card-header>

    <ion-card-content>
      <ion-item fill="outline" class="ion-margin-bottom">
        <ion-label position="floating">Usuario</ion-label>
        <ion-input type="text" [(ngModel)]="username" placeholder="admin"></ion-input>
      </ion-item>

      <ion-item fill="outline" class="ion-margin-bottom">
        <ion-label position="floating">Contraseña</ion-label>
        <ion-input type="password" [(ngModel)]="password" placeholder="123"></ion-input>
      </ion-item>

      <ion-button expand="block" (click)="handleLogin()" class="ion-margin-top">
        Acceder al Panel
      </ion-button>

      @if (errorMessage) {
        <ion-text color="danger">
          <p class="ion-text-center">{{ errorMessage }}</p>
        </ion-text>
      }
    </ion-card-content>
  </ion-card>
</ion-content>
```

src/app/pages/login/login.page.ts

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import {
  IonContent, IonHeader, IonTitle, IonToolbar, IonCard,
  IonCardHeader, IonCardTitle, IonCardSubtitle, IonCardContent,
  IonItem, IonLabel, IonInput, IonButton, IonText
} from '@ionic/angular/standalone';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.page.html',
  styleUrls: ['./login.page.scss'],
  standalone: true,
  imports: [
    CommonModule, FormsModule,
    IonContent, IonHeader, IonTitle, IonToolbar, IonCard,
    IonCardHeader, IonCardTitle, IonCardSubtitle, IonCardContent,
    IonItem, IonLabel, IonInput, IonButton, IonText
  ]
})
export class LoginPage {
  private authService = inject(AuthService);
  private router = inject(Router);

  public username = '';
  public password = '';
  public errorMessage = '';

  public handleLogin(): void {
    this.errorMessage = '';
    const success = this.authService.login(this.username, this.password);
    if (success) {
      this.router.navigate(['/admin']);
    } else {
      this.errorMessage = 'Credenciales no autorizadas. Use admin / 123';
    }
  }
}
```

Paso 6: Vista y Lógica de Administración Protegida

src/app/pages/admin/admin.page.html

HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="dark">
    <ion-title>Zona Segura: Panel de Control</ion-title>
    <ion-buttons slot="end">
      <ion-button (click)="handleLogout()">Cerrar Sesión</ion-button>
    </ion-buttons>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-card color="light">
    <ion-card-header>
      <ion-card-title>Bienvenido, {{ currentSession()?.username }}</ion-card-title>
      <ion-card-subtitle>Rol: {{ currentSession()?.role }}</ion-card-subtitle>
    </ion-card-header>
    <ion-card-content>
      <p>Has superado la validación del enrutador (<code>authGuard</code>).</p>
      <p>La sesión está almacenada activamente en <code>sessionStorage</code>.</p>
    </ion-card-content>
  </ion-card>
</ion-content>
```

src/app/pages/admin/admin.page.ts

TypeScript

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Router } from '@angular/router';
import {
  IonContent, IonHeader, IonTitle, IonToolbar,
  IonCard, IonCardHeader, IonCardTitle, IonCardSubtitle,
  IonCardContent, IonButtons, IonButton
} from '@ionic/angular/standalone';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-admin',
  templateUrl: './admin.page.html',
  styleUrls: ['./admin.page.scss'],
```

```
  standalone: true,
  imports: [
    CommonModule,
    IonContent, IonHeader, IonTitle, IonToolbar,
    IonCard, IonCardHeader, IonCardTitle, IonCardSubtitle,
    IonCardContent, IonButtons, IonButton
  ]
})
export class AdminPage {
  private authService = inject(AuthService);
  private router = inject(Router);

  public currentSession = this.authService.currentUser;

  public handleLogout(): void {
    this.authService.logout();
    this.router.navigate(['/login']);
  }
}
```

3. Demostración y Verificación Técnica

3.1. Inspección de la Persistencia en las Developer Tools

1. Ejecuta la aplicación:
   Bash
   ionic serve

2. Abre las herramientas de desarrollo del navegador (F12 o Ctrl + Shift + I).
3. Ve a la pestaña **Application** (en Chrome/Edge) o **Almacenamiento** (en Firefox).
4. En el panel izquierdo, despliega **Session Storage** y haz clic sobre http://localhost:8100.
5. Inicia sesión con admin y 123:
   - Observa cómo se crea inmediatamente la clave DAM2_AUTH_SESSION.
   - Su contenido será:
     JSON
     {"username":"admin","role":"admin","isAuthenticated":true}

3.2. Prueba de Ataque por Acceso Directo de URL

1. Pulsa el botón **Cerrar Sesión**.
2. Verifica en las DevTools que la clave DAM2_AUTH_SESSION ha sido eliminada.
3. En la barra de direcciones del navegador, escribe manualmente:
   Plaintext
   http://localhost:8100/admin

4. Presiona **Enter**.
5. **Resultado comprobable:** La URL vuelve de inmediato a /login. La vista de administración nunca se monta en el DOM ni se visualiza contenido, ya que authGuard interceptó el evento de enrutamiento y retornó el UrlTree de redirección.

4. Implementaciones propuestas

Enunciado del Ejercicio

Sobre la base del proyecto desarrollado:

1. **Ampliación de Roles y Control de Acceso Granular:**
   - Modifica AuthService para permitir dos tipos de usuarios:
     - admin / 123 (Rol: 'admin')
     - operador / 123 (Rol: 'user')
2. **Generación de una Nueva Ruta:**
   - Crea una página pages/user-dashboard.
   - Crea una factoría de guards roleGuard(allowedRole: string) que valide si el usuario cuenta con el rol requerido.
   - Si un usuario con rol user intenta escribir directamente en la barra /admin, el sistema debe denegar el acceso y redirigirlo a /user-dashboard.